7.4 The maze

Specifications

A file includes a labyrinth with the following specifications:

- The first line of the file specifies the number of row R and the number of columns C of a matrix
- The following R lines specify the matrix rows (each one with C characters), where each
  - `'@'` indicates the initial position of a human being.
  - `' '` represents corridors (empty cells).
  - `'*'` represents walls (full cells).
  - `'#'` represents exit points.

  Supposed only one person is present in the maze in the initial configuration.

The following is a correct example of such a maze:

```
12 10
**********
*        *
*   **@* *
* * ** * *
* ****** *
*   **   *
* *    ***
* ** * ***
* *  * * *
* * ** * *
*   **   *
******#***
```

Write three recursive functions to find:

- One escape path
- All escape paths
- The shortest escape path

from the maze. Print out the solution (or all solutions) on standard output.

Solution

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #define MAX 100
6
7  #define EMPTY ' '
8  #define START '@'
9  #define STOP  '#'
10 #define PATH  '$'
11 #define DONE  '.'
```

```
12 const int xOff[4] = { 0, 1, 0, -1};
13 const int yOff[4] = {-1, 0, 1,  0};
14
15 void display(char **, int);
16 int move_r_one(char **, int, int, int, int);
17 int move_r_all(char **, int , int, int, int, int, int);
18 int move_r_best(char **, int, char **, int, int, int, int, int);
19 FILE *util_fopen(char *, char *);
20 void *util_malloc(int);
21 char *util_strdup(char *);
22
23 int main (int argc, char *argv[]) {
24   int r=-1, c=-1, i, j, nr, nc, res;
25   char **mazeCurr, **mazeBest;
26   char line[MAX];
27   FILE *fp;
28
29   if (argc < 2) {
30     fprintf(stdout, "Error: missing parameter.\n");
31     fprintf(stdout, "Run as: %s <maze_file>\n", argv[0]);
32     exit(EXIT_FAILURE);
33   }
34
35   fp = util_fopen(argv[1], "r");
36   fgets(line, MAX, fp);
37   sscanf(line, "%d %d", &nr, &nc);
38   mazeCurr = (char **)util_malloc(nr * sizeof(char *));
39   mazeBest = (char **)util_malloc(nr * sizeof(char *));
40   for (i=0; i<nr; i++) {
41     fgets(line, MAX, fp);
42     mazeCurr[i] = util_strdup(line);
43     mazeBest[i] = util_strdup(line);
44     for (j=0; j<nc; j++) {
45       if (mazeCurr[i][j] == START) {
46         r = i;
47         c = j;
48       }
49     }
50   }
51
52   if (r<0 || c<0) {
53     fprintf(stdout, "Error: no startig position found!\n");
54     exit(EXIT_FAILURE);
55   }
56
57   // Find one solution
58   fprintf(stdout, "Find one solution:\n");
59   mazeCurr[r][c] = EMPTY;
60   res = move_r_one(mazeCurr, nr, nc, r, c);
61   if (res == 1) {
62     mazeCurr[r][c] = START;
63     display(mazeCurr, nr);
64   } else {
65     fprintf(stdout, "NO solution found!\n");
66   }
67   // Clean matrix
68   for (i=0; i<nr; i++) {
69     for (j=0; j<nc; j++) {
70
```

```
71          if (mazeCurr[i][j]==PATH || mazeCurr[i][j]==DONE) {
72            mazeCurr[i][j] = EMPTY;
73          }
74        }
75      }
76
77      // Find all solutions
78      fprintf(stdout, "Find all solutions:\n");
79      mazeCurr[r][c] = EMPTY;
80      res = move_r_all(mazeCurr, nr, nc, r, c, r, c);
81
82      // Find the best solution
83      fprintf(stdout, "Find the best solutions:\n");
84      mazeCurr[r][c] = EMPTY;
85      res = move_r_best(mazeCurr, 0, mazeBest, nr*nc, nr, nc, r, c);
86      if (res > 0) {
87        fprintf(stdout, "Solution:\n");
88        mazeBest[r][c] = START;
89        display(mazeBest, nr);
90      } else {
91        fprintf(stdout, "NO solution found!\n");
92      }
93
94      for (r=0; r<nr; r++) {
95        free(mazeCurr[r]);
96        free(mazeBest[r]);
97      }
98      free(mazeCurr);
99      free(mazeBest);
100     return EXIT_SUCCESS;
101   }
102
103   /*
104    *  find one (the first) solution
105    */
106   int move_r_one (char **mazeCurr, int nr, int nc, int row, int col) {
107     int k, r, c;
108
109     if (mazeCurr[row][col] == STOP) {
110       return 1;
111     }
112     if (mazeCurr[row][col] != EMPTY) {
113       return 0;
114     }
115
116     mazeCurr[row][col] = PATH;
117     for (k=0; k<4; k++) {
118       r = row + xOff[k];
119       c = col + yOff[k];
120       if (r>=0 && r<nr && c>=0 && c<nc) {
121         if (move_r_one(mazeCurr, nr, nc, r, c) == 1)
122           return 1;
123       }
124     }
125     mazeCurr[row][col] = DONE;
126
127     return 0;
128   }
129
```

```
/*
 *  find all solutions
 */
int move_r_all (
  char **mazeCurr, int nr, int nc, int row, int col, int row0, int col0
) {
  int k, r, c;
  static int solN = 0;

  if (mazeCurr[row][col] == STOP) {
    solN++;
    fprintf(stdout, "Solution #%d:\n", solN);
    mazeCurr[row0][col0] = START;
    display(mazeCurr, nr);
    mazeCurr[row0][col0] = EMPTY;
    return 1;
  }
  if (mazeCurr[row][col] != EMPTY) {
    return 0;
  }

  mazeCurr[row][col] = PATH;
  for (k=0; k<4; k++) {
    r = row + xOff[k];
    c = col + yOff[k];
    if (r>=0 && r<nr && c>=0 && c<nc) {
      move_r_all(mazeCurr, nr, nc, r, c, row0, col0);
    }
  }
  mazeCurr[row][col] = EMPTY;

  return 0;
}

/*
 *  find the best solution
 */
int move_r_best (
  char **mazeCurr, int stepCurr, char **mazeBest,
  int stepBest, int nr, int nc, int row, int col
) {
  int k, r, c;

  /* Avoid stopping recursion to find ALL solutions */
  if (stepCurr >= stepBest) {
    return stepBest;
  }
  if (mazeCurr[row][col] == STOP) {
    /* STOP recurring to print-out only ONE solution */
    /* Print all matrices to have ALL solutions */
    if (stepCurr < stepBest) {
      stepBest = stepCurr;
      for (r=0; r<nr; r++) {
        for (c=0; c<nc; c++) {
          mazeBest[r][c] = mazeCurr[r][c];
        }
      }
    }
    return stepBest;
```

```
189     }
190     if (mazeCurr[row][col] != EMPTY) {
191       return stepBest;
192     }
193
194     mazeCurr[row][col] = PATH;
195     for (k=0; k<4; k++) {
196       r = row + xOff[k];
197       c = col + yOff[k];
198       if (r>=0 && r<nr && c>=0 && c<nc) {
199         stepBest = move_r_best (
200           mazeCurr, stepCurr+1, mazeBest, stepBest, nr, nc, r, c);
201       }
202     }
203     mazeCurr[row][col] = EMPTY;
204
205     return stepBest;
206   }
207
208   /*
209    *  write the maze scheme on screen
210    */
211   void display (char **maze, int nr) {
212     int i;
213
214     for (i=0; i<nr; i++) {
215       fprintf(stdout, "%s", maze[i]);
216     }
217   }
218
219   /*
220    *  fopen (with check) utility function
221    */
222   FILE *util_fopen (char *name, char *mode) {
223     FILE *fp=fopen(name, mode);
224
225     if (fp == NULL) {
226       fprintf(stdout, "File open error (file=%s).\n", name);
227       exit(EXIT_FAILURE);
228     }
229
230     return fp;
231   }
232
233   /*
234    *  malloc (with check) utility function
235    */
236   void *util_malloc (int size) {
237     void *ptr=malloc(size);
238
239     if (ptr == NULL) {
240       fprintf(stdout, "Memory allocation error.\n");
241       exit(EXIT_FAILURE);
242     }
243
244     return ptr;
245   }
246
247   /*
```

```
248   * strdup (with check) utility function
249   */
250  char *util_strdup (char *src) {
251    char *dst=strdup(src);
252
253    if (dst == NULL) {
254      fprintf(stdout, "Memory allocation error.\n");
255      exit(EXIT_FAILURE);
256    }
257
258    return dst;
259  }
```